

《计算概论》 程序设计部分

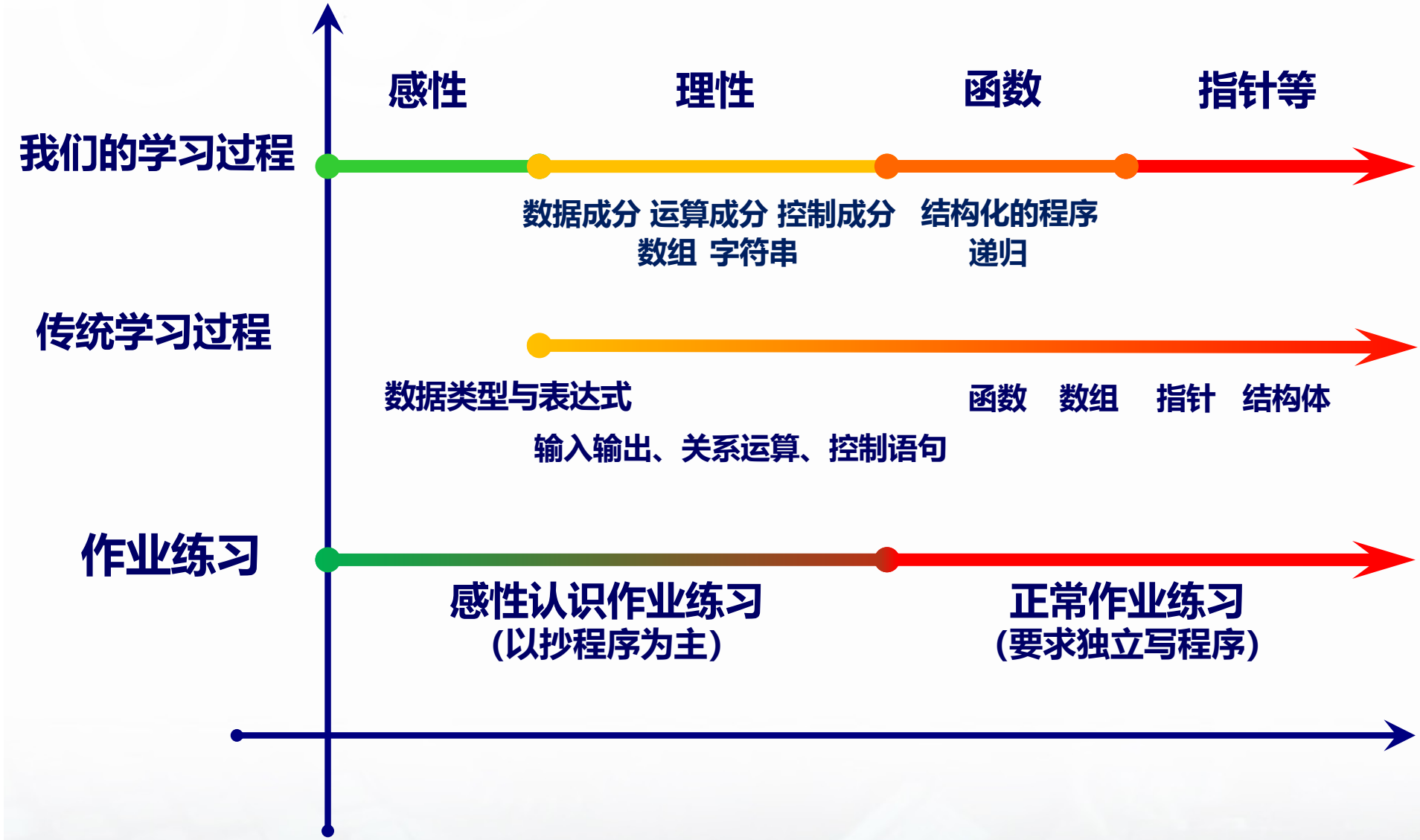
C 语言的构成成分——运算成分

董 豪

北京大学 计算机学院 前沿计算研究中心

hao.dong@pku.edu.cn

再谈，我们的进度安排


感性
理性
函数
指针等
我们的学习过程
数据成分 运算成分 控制成分 结构化的程序
数组 字符串
递归
传统学习过程
数据类型与表达式
函数 数组 指针 结构体
输入输出、关系运算、控制语句
作业练习
感性认识作业练习
(以抄程序为主)
正常作业练习
(要求独立写程序)

问题1：

是不是 无论我们在程序里写什么“单词”，计算机都能明白？

问题2：

是不是 无论我们在程序里写什么“数”和“计算符号”，计算机都能明白？

问题3：

世界上可能要用“程序来表达的逻辑”纷繁复杂，程序设计语言里面得有多少种“句式”才能够用？

问题2：

是不是 无论我们在程序里写什么“数”和“计算符号”，计算机都能明白?

- NO!
- 计算机只能“看懂”某些类型的数据，这些“数据的类型”和相应的“**操作符号**”也是定义好的。

程序设计语言的构成

- **语言的种类千差万别，但是，一般说来，基本成分不外四种：**
  - **数据成分**，用以描述程序中所涉及的数据；
  - **运算成分**，用以描述程序中所包含的运算；
  - **控制成分**，用以表达程序中的控制构造；
  - **传输成分**，用以表达程序中数据的传输；

——计算机科学技术百科全书



表达式

- 表达式是 C/C++ 运算成分的基本单位。
  - 表达式是指程序中由运算符和它们的操作数组成等所组成的计算式。如：

```
a * b + c
123 < 10
'a' * 3.14f == 1
a = b
```

  - 操作数可以是变量、字面量和表达式。
  - 和数学上类似地，任何表达式都有一个结果。有时我们称这个结果为它的值或者它的返回值。



C++ 中的运算符

- C++ 的运算符范围很宽。

自增/减运算符	++ --
C风格类型转换运算符	(type) type() type{}
C++风格类型转换运算符	const_cast static_cast dynamic_cast reinterpret_cast
函数调用运算符	a()
下标运算符	a[]
成员访问运算符	. -> .* ->*
一元加/减运算符	+ -
取/解地址运算符	&a *a
取大小运算符	sizeof
动态内存分配运算符	new new[] delete delete[]
算术运算符	+ - * / %
关系运算符	< <= > >= == != <=>
位运算符	& ^ | << >> ~a
逻辑运算符	&& || !a
条件运算符	a?b:c
赋值运算符	= += -= *= /= %= <<= >>= &= ^= |=
逗号运算符	,
协程运算符	co_yield co_await
其它	throw typeid sizeof... noexcept alignof ::

使用说明

- 注意：
  - 整型进行运算，结果仍为整型
    - `5 / 3` 的结果仍是整数，小数部分被忽略
  - 浮点类型进行运算，结果为浮点类型
    - `5.3 / 3` 或 `5 / 3.0` 的结果为 `double` 型
  - 浮点数向整数舍入的方向由编译器决定
    - 一般是“向零取整”，即抹去小数部分



赋值运算



赋值运算符

- “=” 赋值运算符
  - 给赋值号左边的变量赋予数值
- 在变量定义的同时可以为变量初始化。如:
  - int a = 3;

    相当于:

    int a; a = 3;
  - int a, b, c = 5

    表示只给 c 初始化。相当于

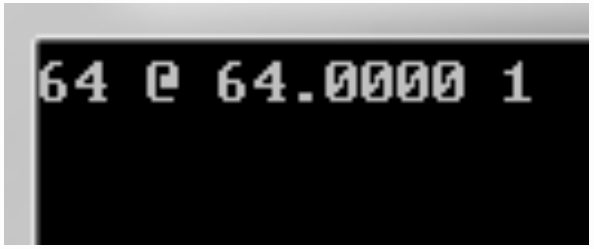
    int a, b, c;

    c = 5;
  - int a = b = c = 5; (这是对的吗?)
    - 等于int a = (b = (c = 5));

类型转换

- **要点一：两边类型不同**
  - 若 = 两边的类型不一致，赋值时要进行类型转换。
  - 不管 = 右边的操作数是什么类型，都要转换为 = 左边的类型

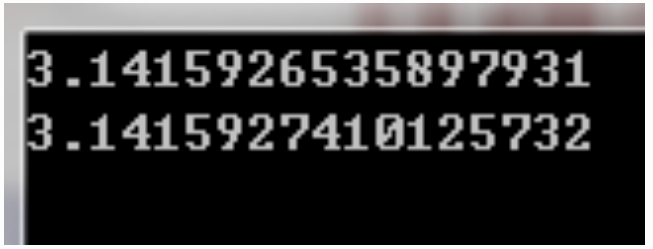
```
int main()
{
        int int_i = 64.12345;
        char char_i = int_i;
        float float_i = char_i;
        bool bool_i = float_i;
        cout << showpoint << int_i << " " << char_i <<
                                " " << float_i << " " << bool_i << endl;
        return 0;
}
```



自动类型转换（1）

- 如果=右边的操作数具有**较高级别**的类型，则在类型转换时，进行**截断取舍**，可能会损失精度!

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
        double double_i = 3.14159265358979323846;
        cout << setprecision(30) << double_i << endl;
        float float_i = double_i;
        cout << float_i << endl;
        return 0;
}
```



自动类型转换（2）

- 如果=右边的操作数类型级别较低，则在类型转换时，采用补齐方式，不会损失精度。

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
        float float_i = 3;
        cout << showpoint << float_i << endl;
        double double_i = float_i;
        cout << setprecision(20) << double_i << endl;
        return 0;
}
```



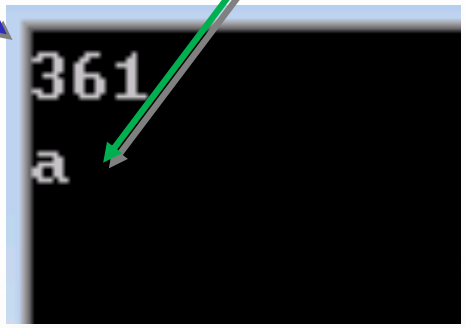
赋值运算符

- 要点二：长数赋给短数
  - 截取长数的低n位送给短数




```
int main()
{
        char char_a  = ' ';
        int int_i = 0x361;
        cout << hex << int_i << endl;
        char_a = int_i;
        cout << char_a << endl;
        return 0;
}
```





赋值运算符

- 举例：short = long
  - 截取长整型数的低16位送给short字符。
  - 如果最高位为1，则得到负数，否则得到正数;




```
int main()
{
        long int long_i = 0x2AAAAAAA;
        cout << long_i << endl;
        short short_j = long_i;
        cout << hex << short_j <<endl;
        cout << dec << short_j << endl;
        return 0;
}
```




```
715827882
aaaa
-21846
```

赋值运算符

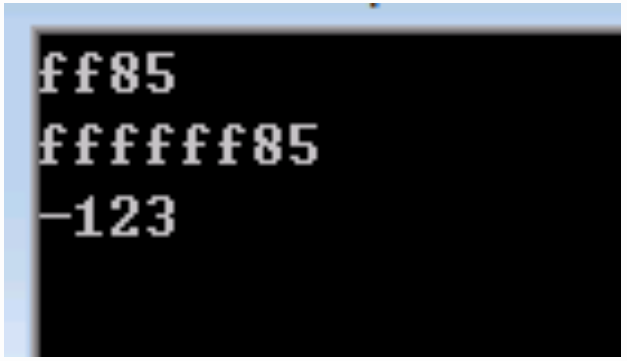
- 要点三：短数赋给长数
  - 最好处理的情况！原来是什么数，现在还是什么数！
  - short int a = -1;　　int b = a;

计算机的处理过程：

- 若short型数为无符号数：
  - short 型16位到long型低16位，long型高16位补0；
- 若 short型数为有符号数：
  - short型16位到 long型低16位；
  - 若short型最高位为0，则long型高16位补0；
  - 若short型最高位为1，则long型高16位补1；

示例

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
        short short_i = -123;
        cout << hex << short_i << endl;
        int  int_j = short_i;
        cout << hex << int_j <<endl;
        cout << dec << int_j << endl;
        return 0;
}
```



赋值运算符

- 要点四：符号位的赋值处理

signed



直接搬运

unsigned



赋值运算符

- 要点四：符号位的赋值处理
  - 也很好处理的情况！直接搬运！
  - 直接赋值，不管符号位还是数字位！

例如：

- unsigned = int 或long
  - 直接赋值，符号位当作数字。
- int = unsigned int
  - 直接赋值，数字当作符号位。

signed

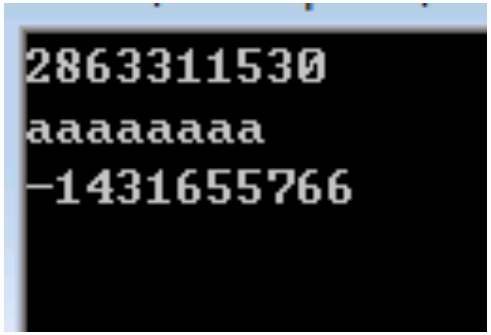
1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

unsigned

1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

示例

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
        unsigned int unsigned_int_i = 0xAAAAAAAA;
        cout << unsigned_int_i << endl;
        signed int  signed_int_j = unsigned_int_i;
        cout << hex << signed_int_j << endl;
        cout << dec << signed_int_j << endl;
        return 0;
}
```



signed

1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

unsigned

1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

赋值运算四个要点总结

- 当：两边类型不同
  - 自动完成类型转换！
- 当：长数赋给短数
  - 截取长数的低位送给短数！
- 当：短数赋给长数
  - 原来是什么数，现在还是什么数！
- 当：符号位的赋值处理
  - 直接赋值，不管符号位还是数字位！

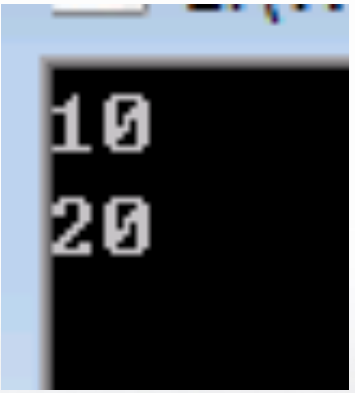
赋值运算第一点附加说明：表达式

- 什么是表达式
  - 程序中由运算符、操作数和括号组成等所组成的计算式，是计算求值的基本单位。

a * b + c;　　123 < 10;　　'a'*3.14f == 1;　　a = b;

  - 表达式是有“值”的，赋值语句也不例外；

```
int main()
{
        int i = 0;
        cout << (i = 10) << endl;
        cout << (i = i + i) << endl;
        return 0;
}
```

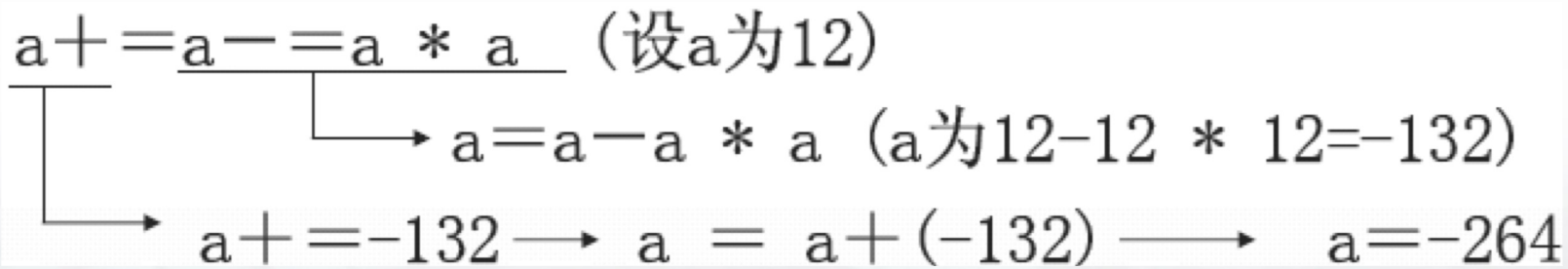


赋值运算第二点附加说明：复合赋值运算

- 复合的赋值运算
  - 在赋值符号前加上其它运算符号则构成复合赋值运算；
  - 例如：
    - a + = 3; 等价于a = a + 3;
    - x * = y + 8; 等价于x = x * (y + 8);
    - x % = 3; 等价于x = x % 3;

赋值运算第三点附加说明：连续赋值运算

- 连续的赋值运算
  - 自右而左的结合顺序
    - a = b = 5;　　//a, b均为5
    - a = b = c = 5;　　//a, b, c均为5
      - int a=b=c=5;　　//编译错！
  - a = (b = 4) + (c = 6); //a为10, b为4, c为6
- 举例：





算术运算

算术表达式

+ -
* / %

- 算术运算符
  - 加法、减法、乘法和除法
  - % 是模运算，即，求余运算，必须是整数
    - 如 7 % 4 的值为 3。
- 算术表达式
  - 包含算术运算符的表达式，表达式的值为运算的结果。
  - 写法：

左侧操作数 算术运算符 右侧操作数

a * b

算术运算符和算术表达式

- 算术运算符和算术表达式
  - 基本的算术运算 + 、 - 、 *、 / 、 %
    - % 是模运算，即，求余运算，必须是整数
      - 如 7 % 4 = 3
- 注意：
  - 整数运算，结果仍为整数
    - 5 / 3 的结果仍是整数，小数部分被忽略
  - 实数运算，结果为double型
    - 5.3 / 3 或 5 / 3.0 的结果为double型
  - 舍入的方向随编译器的不同而不同

算术运算符和算术表达式

- 例子

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
        double a = 5/3;
        double b = 5/3.0;
        cout << setprecision(20) << a <<
endl;
        cout << b << endl;
        return 0;
}
```

```
1
1.6666666666666667407
```

算术表达式

- 算术运算符的优先级

  大 （ ）
  中 * / %
  小 + -

  - 在同一级别中，采取由左至右的结合方向
    - 如：a – b + c 相当于(a – b) + c
    - 如：a % b * c / d 相当于(((a % b)*c))/d
  - 当数据类型非常复杂时
    - 如：123%s + (i+‘@’) + i*u – f/d

    其中 short s; int i; float f; double d; unsigned u;

算术表达式

- 剪刀法求表达式的值

short s; int i; float f; double d; unsigned u;

123%s + (i+'@') + i*u + f/d

第三刀　　第二刀　第一刀

int　　int　　unsigned

int

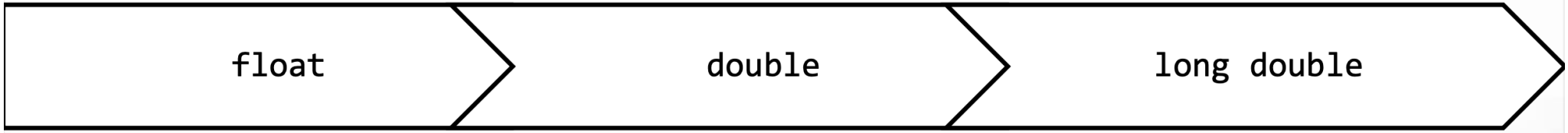
unsigned　　double

double

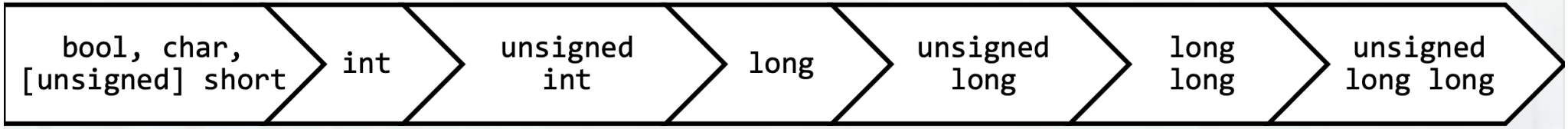
计算过程中的类型转换

整数类型×浮点类型：转换为浮点类型

浮点类型×浮点类型：按此顺序“提升”



整数类型×整数类型：按此顺序“提升”





算术表达式之

自增、自减运算

算术表达式

- 自增、自减运算符：使变量的值加1或减1
  - i + + ,　i - - （回顾）
    - i++ 等于 i=i+1
    - 在使用i之后，再将i的值加（减）1
  - + + i,　- - i
    - 在使用i之前，先将i的值加（减）1



算术表达式

- 例如：i 的值为3，则
  - j = + + i;　　// i为4，j为4
  - j = i + +;　　// i为4，j为3
  - cout<<++i;　　//输出4，i为4
  - cout<<i++;　　//输出3，i为4



自增、自减运算符

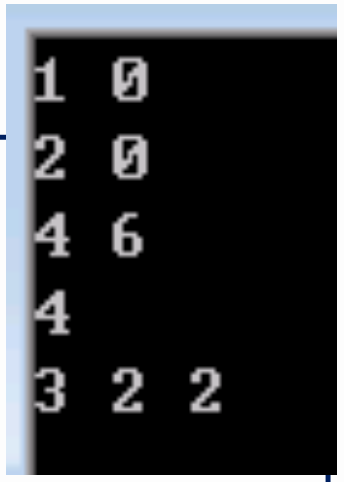
- 举例：
  - 设i的值为3；则

```
cout << -i++ << endl;      // 输出-3，i为4
cout << -++i << endl;      // 输出-4，i为4
cout << (-i)++ << endl;    // 不可以
cout << ++i++ << endl;     // 不可以
```

- 注意：
  - ++和--只能用于变量

自增、自减运算符

```
int main()
{
    int a = 0, b = 0, c = 2, d = 0, e = 2, f = 2;
    cout << a << " " << a++ << " " << endl;
    cout << ++b << " " << b++ << " " << endl;
    cout << c << " " << (c++) + (++c) << " " << endl;
    cout << (d = f++) + (e = f) << endl;
    cout << f << " " << d << " " << e << endl;
    return 0;
}
```



Visual C++

不要纠结于此

```c
#include <stdio.h>
int main(int argc, char const *argv[])
{
    int w,x,y,z;
    x=4,w=5;
    y=w++*w++*w++;
    z=--x*--x*--x;
    printf("%d,%d,%d,%d\n",w,x,y,z );
    return 0;
}
```

Mac+XCode

```
mac — 1 — 80×24
Last login: Sat Mar 14 14:31:44 on ttys000
promote:~ mac$ /Users/mac/Desktop/1 ; exit;
8,1,210,6
logout

[进程已完成]
```

XP+VC

```
"C:\Documents and Settings\Adm
8,1,125,4
Press any key to continue
```

Expression : MESS

Communication : MESS

WIN7+DEV C

```
C:\Users\king\Desktop\未命名1.exe
8,1,210,4

--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

DO NOT
DO SUCH FOOLISH THING!!!

dl;

的运行结果也不同。

小结运算符优先级

- **运算符的优先级：**

优先级	运算符	说明	结合方向
2	a++ a--	**后缀自增与自减**	→
3	++a --a	**前缀自增与自减**	←
5	a*b a/b a%b	乘法、除法与取余	→
6	a+b a-b	加法与减法	→
16	= += -= *= /= %=	**简单与复合赋值**	←



关系运算

关系表达式

<	<=
>	>=
==	!=

- C++提供了7种关系运算符，这是其中的6种：
  - < (小于)
  - <= (小于或等于)
  - > (大于)
  - >= (大于或等于)
  - == (等于)
  - != (不等于)



关系表达式

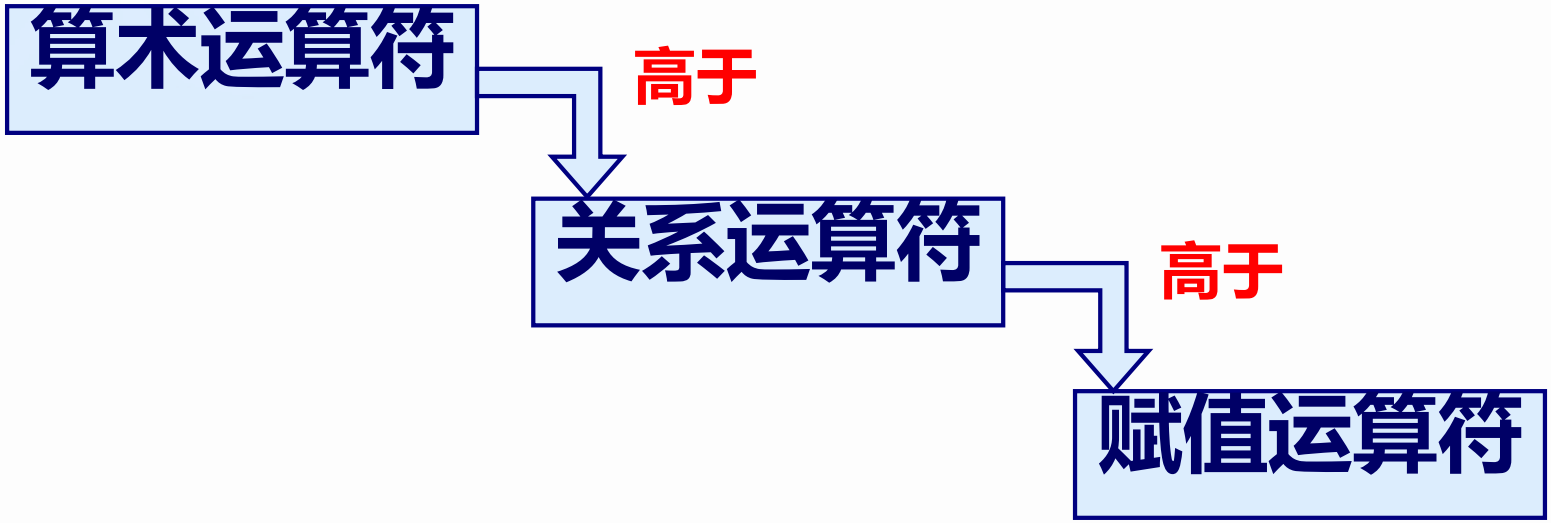
- **关系表达式：**
  - **写法：**

    左侧操作数 关系运算符 右侧操作数

  - **结果：**
    - **`true` 或者 `false`，表明关系是否成立**

```
int a = 3, b = 4;
a > b;    // 值为 false
a != b;   // 值为 true
a == b;   // 值为 false
```

运算符的优先级



- 例如：
  - 1 + 2 % 3 * 4 > 5 / 6 – 7
  - 1 + 2 % (3 * 4) > (5 / 6 – 7) = = 8
  - X = 1 + 2 % 3 * 4 > 5 / 6 – 7 = = 8
  - 1 > 2 = = 3 > 4

小结运算符优先级

优先级	运算符	说明	结合方向
2	a++ a--	后缀自增与自减	→
3	++a --a	前缀自增与自减	←
5	a*b a/b a%b	乘法、除法与取余	→
6	a+b a-b	加法与减法	→
9	< <= > >=	（不）小于与大于	→
10	== !=	等于与不等于	→
16	= += -= *= /= %=	简单与复合赋值	←

```
1 + 2 % 3 * 4 > 5 / 6 - 7
1 + 2 % (3 * 4) > (5 / 6 - 7) == 8
X = 1 + 2 % 3 * 4 > 5 / 6 - 7 == 8
1 > 2 == 3 > 4
```

勤加括号！



实践Tips：

多用括号控制优先级

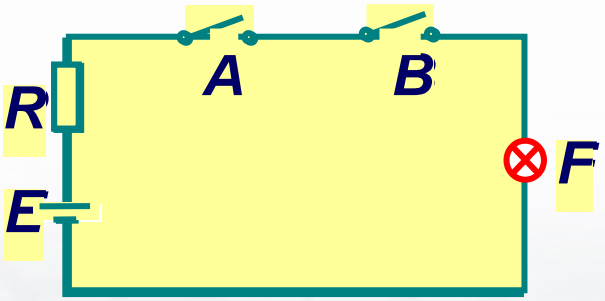


逻辑运算

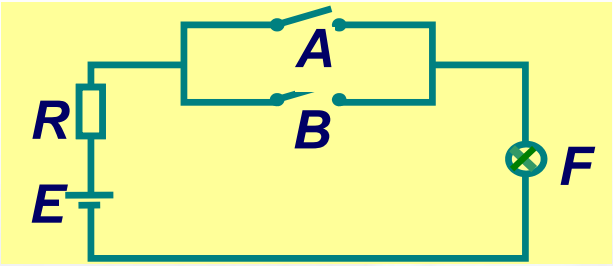
逻辑表达式

&& || !

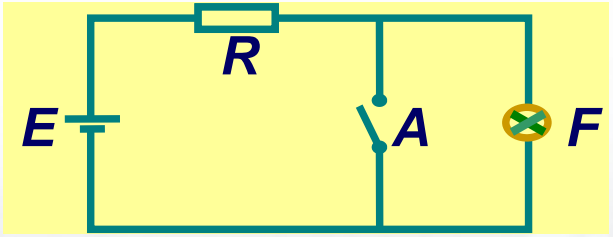
- 三种逻辑运算符，针对布尔型：
  - 逻辑与 &&
    - 若 A、B 都为真，A与B 才为真。
  - 逻辑或 ||
    - 若 A、B 之一为真，则 A或B 就为真。
  - 逻辑非 !
    - 若 A 为真，则 非A 为假。



与门



或门



非门



逻辑表达式

■ 逻辑表达式

左侧操作数 || *右侧操作数*

左侧操作数 && *右侧操作数*

!*操作数*

- ◆ **操作数不一定是布尔型。它们将进行隐式转换：**
  - **当操作数非 0 值，隐式转换为 true ，**
  - **当操作数是 0 值，隐式转换为 false 。**
- ◆ **优先次序：**
  - **！ 高于 && 高于 || ，即 非 优先级最高。**

逻辑表达式的值

- 以0代表“假”；以非0代表“真”。
  - 若 a = 4 ，则 ! a 的值为 0。
  - 若 a = 4，b = 5，则 a && b 的值为1。
  - 若 a = 4，b = 5，则 a || b 的值为1。
  - 若 a = 4，b = 5，则 !a || b 的值为1。
  - 表达式 4 && 0 || 2 的值为1。

逻辑表达式的值（练习）

- 以0代表“假”；以非0代表“真”。
  - 若 a = -1，则 ! a 的值为 ?
  - 若 a = 0，b = 5，则 a && b 的值为?
  - 若 a = -1，b = 0，则 a || b 的值为?
  - 若 a = 0，b = 0，则 !a || b 的值为?



运算符优先级

优先级	运算符	说明	结合方向
2	a++ a--	后缀自增与自减	→
3	++a --a !	前缀自增与自减，**逻辑非**	←
5	a*b a/b a%b	乘法、除法与取余	→
6	a+b a-b	加法与减法	→
9	< <= > >=	（不）小于与大于	→
10	== !=	等于与不等于	→
14	&&	**逻辑与**	→
15	||	**逻辑或**	→
16	= += -= *= /= %=	简单与复合赋值	←

a > b && x > y **可写成** (a > b) && (x > y)

a == b || x == y **可写成** (a == b) || (x == y)

!a || a > b **可写成** (!a) || (a > b)

逻辑运算符优先级

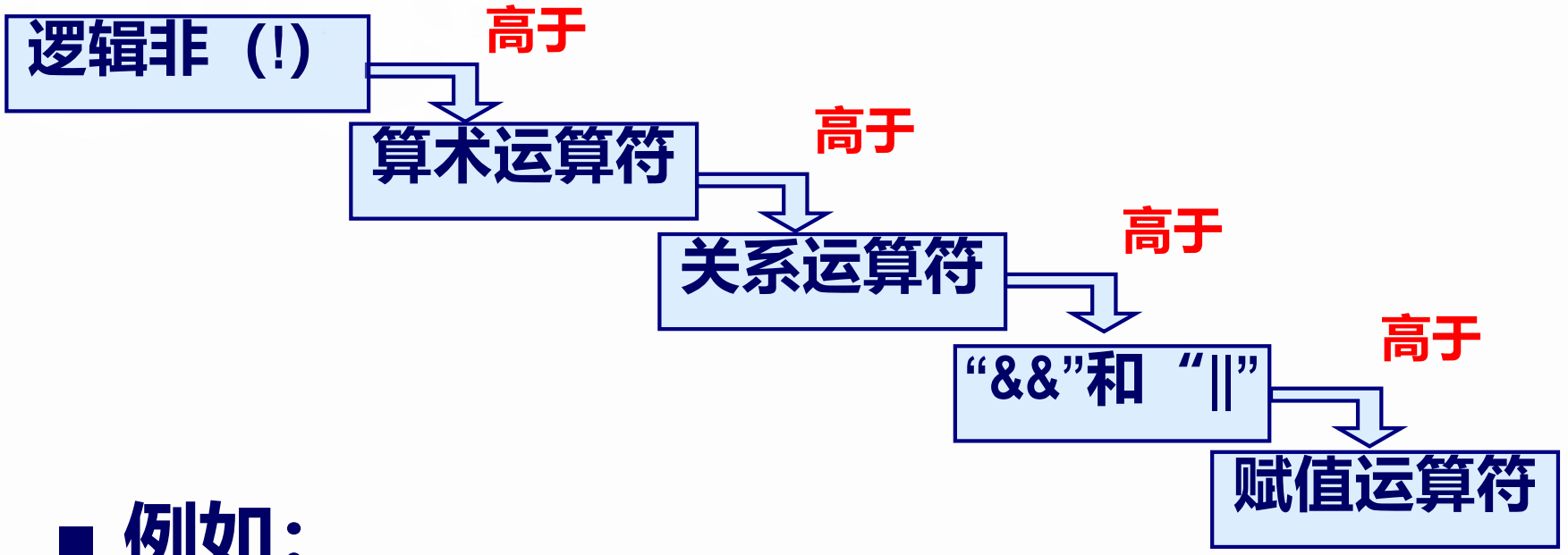
- 逻辑表达式
  - 一个逻辑表达式中若包含多个逻辑运算符，则按以下的优先次序：
    - ！(非)→&& (与)→|| (或)，即“!” 优先级最高。
  - 例如：

```
!a&&b||c;
! a && b || x > y && c;
! a && b || x > y && c + 1;
```

思考题

```
#include <iostream>
using namespace std;
int main() {
    int a = 0, b = 0;
    a = 5 > 3 && 2 || 8 < 4 - (b = !0);
    cout << a << " " << b;
    return 0;
}
```

混合运算的优先级



- 例如：
  - a > b && x > y可写成(a > b) && (x > y)
  - a == b || x == y可写成(a ==b) || (x == y)
  - ! a || a > b可写成 (! a) || (a > b)

逻辑运算的“短路特性”

- 逻辑表达式求解中，并不总是执行所有的运算
  - 只有在必须执行下一个逻辑运算符才能求出表达式的解时，才执行该运算符！
- 对于表达式 a && b && c
  - 只要 a 为 false ，就不必判断 b 和 c ；
  - 只有 a 为 true 时，才需要判别 b 的值；
  - 只有 a 和 b 都为 true 的情况下才判别 c 。
- 对于表达式 a || b || c
  - 只要 a 为 true ，就不必判断 b 和 c ；
  - 只有 a 为 false 时，才判别 b ；
  - 只有 a 和 b 都为 false 才判别 c 。

逻辑运算的“短路特性”举例

```
#include <iostream>
using namespace std;
int main() {
    int i = 0, a = 1, b = 2, c = 3;
    i = ++a || ++b || c++;
    cout << i << " " << a << " "
         << b << " " << c << endl;
    return 0;
}
```

很复杂而且没什么用的示例

- 操作数不一定是布尔型。它们将进行隐式转换：
  - 当操作数非 0 值，隐式转换为 true ，
  - 当操作数是 0 值，隐式转换为 false 。
- 优先次序：
  - ！ 高于 && 高于 || ，即 非 优先级最高。

```
#include <iostream>
using namespace std;
int main() {
    bool a = false, b = true;
    a = 8 > 4 - (b = !'c') && 5 > 3 + 'a' % 6 == 'b';
    cout << a << " " << b;
    return 0;
}
```

1 3 2 5 4

```
0 0
```

应用示例

- 问题：要判别某一年year是否闰年。闰年的条件是符合下面二者之一：
  - ①能被4整除，但不能被100整除。
  - ②能被100整除，又能被400整除。

```cpp
#include <iostream>
using namespace std;
int main() {
    int year = 0;
    cin >> year;
    if (year % 4 == 0) {
        if (year % 100 == 0) {
            if (year % 400 == 0)
                cout << "Y";
            else
                cout << "N";
        } else
            cout << "Y";
    } else
        cout << "N";
    return 0;
}
```

问：有没有更简单的方法?



应用示例

- 问题：要判别某一年year是否闰年。闰年的条件是符合下面二者之一：
  - ①能被4整除，但不能被100整除。
  - ②能被100整除，又能被400整除。
- 求解：
  - 可以用如下逻辑表达式来判别year是否为闰年：

```
(year % 4 == 0 && year % 100 != 0) || year % 400 == 0
```

  - 也可以用如下逻辑表达式判别year是否非闰年：

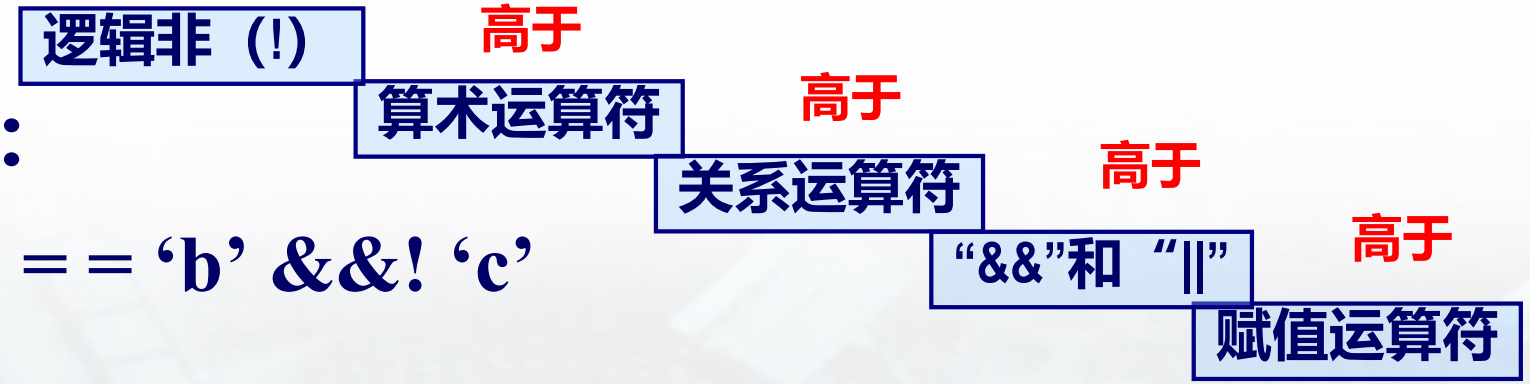
```
(year % 4 != 0) || (year % 100 == 0 && year % 400 != 0)
```

运算对象的扩展

- 逻辑运算符两侧可以是任何类型
  - 如：字符型、实型或指针型等；
  - 系统最终以 0 和 非0 来判定它们，例如

    'c' && 'd'

    - ‘c’和 ‘d’的ASCII值都不为0，按“真”处理。
- 思考：
  - ‘a’ = = ‘b’ &&! ‘c’





今天学习了

- C 语言的运算符范围很宽
  - 求字节数运算符: sizeof
  - 下标运算符 [ ]
  - 赋值运算符 =
  - 算术运算符 + - * / %
  - 关系运算符 < > == >= <= !=
  - 逻辑运算符 ! && ||
  - 条件运算符 ? :
  - 逗号运算符 ,
  - 位运算符 >> ~ | ^ &
  - 指针运算符 *, &
  - 强制类型转换运算符: (类型)
  - 分量运算符 . →

今天



条件表达式

条件表达式

a?b:c

■ 条件表达式

表达式1 ? *表达式2* : *表达式3*

■ 值说明:

◆如果 *表达式1* 的值为 true ，则以 *表达式2* 的值作为条件表达式的值;

◆否则，以 *表达式3* 的值作为整个条件表达式

例如:

```
max = (a > b) ? a : b;
```

相当于:

```
if (a > b)
    max = a;
else
    max = b;
```

逗号表达式

,

- 逗号运算符：将两个表达式连起来
- 逗号表达式：

表达式1 ， 表达式2

- 下式中是否相同?
  - `x = (a = 3, 6 * 3);`
  - `x = a = 3, 6 * 3;`



运算符优先级

优先级	运算符	说明	结合方向
2	a++ a--	后缀自增与自减	→
3	++a --a !	前缀自增与自减，逻辑非	←
5	a*b a/b a%b	乘法、除法与取余	→
6	a+b a-b	加法与减法	→
9	< <= > >=	（不）小于与大于	→
10	== !=	等于与不等于	→
14	&&	逻辑与	→
15	||	逻辑或	→
16	= += -= *= /= %= a?b:c	简单与复合赋值，**条件**	←
17	,	**逗号**	→

注：条件运算符 ? 与 : 中间如同视有括号。



逗号、条件、强转

今天学习了

- C 语言的运算符范围很宽
  - 求字节数运算符: sizeof
  - 下标运算符 [ ]
  - 赋值运算符 =
  - 算术运算符 + - * / %
  - 关系运算符 < > == >= <= !=
  - 逻辑运算符 ! && ||
  - 条件运算符 ? :
  - 逗号运算符 ,
  - 位运算符 >> ~ | ^ &
  - 指针运算符 *, &
  - 强制类型转换运算符: (类型)
  - 分量运算符 . →

刚刚

现在



强制类型转换

强制（显式）类型转换表达式

- 写法:
  - (*类型说明符*) *表达式*
  - *类型说明符*(*表达式*)
- 说明：结果为转换后的 *表达式* 的值
- 举例:
  - `(double)a;` //将a 的值转换成double类型
  - `int(x + y);` //将x+y的值转换成int类型
  - `float(5 / 3);` //将5/3的值转换成float类型
- 注意:
  - 强制类型转换后，被转换的量的类型并没有发生变化

强制类型转换

```
#include <iostream>

using namespace std;

int main()

{

        float a = 3.14159 ;

        cout << (int)a << endl;

        cout << a << endl;

        return 0;

}
```

```
3
3.14159
```



破案

- 题目
  - 试写一个程序，帮助某地刑侦大队利用以下已经掌握的确切线索，从6个嫌疑人中找出作案人：
    - A，B至少有一人作案；
    - A，E，F 3人中至少有2人参与作案；
    - A，D不可能是同案犯；
    - B，C或同时作案，或与本案无关；
    - C，D中有且仅有1人作案；
    - 如果D没有参与作案，则E也不可能参与作案



解题步骤

- 基本方法
  - 测试所有可能的组合；
  - 对于每种组合，测试是否满足已知条件
  - 若满足，该组合是目标；
- 前提条件
  - 将所有已知条件表达出来
    - 如何表示A是作案者?
  - 找到控制循环的方法



条件表达

- A，B至少有一人作案；
  - `cc1 = A || B;`
- A，E，F 3人中至少有2人参与作案；
  - `cc2 = (A && E)||(A && F)||(E && F)`
- A，D不可能是同案犯；
  - `cc3 = !(A && D)`
- B，C或同时作案，或与本案无关；
  - `cc4 = (B && C)||(!B && !C)`
- C，D中有且仅有1人作案；
  - `cc5 = (C && !D)||(!C && D)`
- 如果D没有参与作案，则E也不可能参与作案
  - `cc6 = D || !E`



程序框架

```
int main() {
    //[...]
    for (A = 0; A <= 1; A++)
        for (B = 0; B <= 1; B++)
            for (C = 0; C <= 1; C++)
                for (D = 0; D <= 1; D++)
                    for (E = 0; E <= 1; E++)
                        for (F = 0; F <= 1; F++) {
                            if (cc1 && /*[...]*/ cc6) {
                                // 打印结果;
                            }
                        }
}
```

总结：运算符优先级

优先级	运算符	说明	结合方向
2	a++ a-- type()	后缀自增与自减， 函数风格转型	→
3	++a --a ! (type)	前缀自增与自减，逻辑非，C 风格转型	←
5	a*b a/b a%b	乘法、除法与取余	→
6	a+b a-b	加法与减法	→
9	< <= > >=	（不）小于与大于	→
10	== !=	等于与不等于	→
14	&&	逻辑与	→
15	||	逻辑或	→
16	= += -= *= /= %= a?b:c	简单与复合赋值，条件	←
17	,	逗号	→

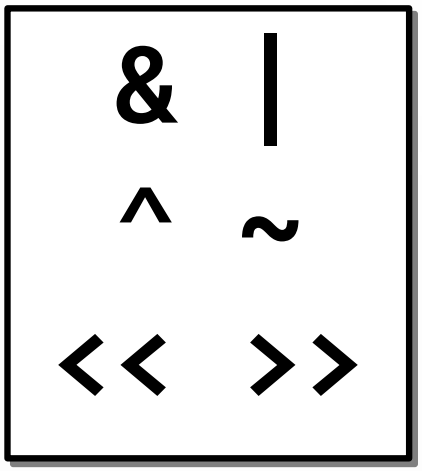
注：条件运算符 ? 与 : 中间如同视有括号。



选讲内容 —— 位运算

（对做题很有用）

位运算表达式



- 位运算
  - 所谓位运算是指进行二进制位的运算。
- C++语言中的位运算符
  - 按位与（&） 双目运算符
  - 按位或（|） 双目运算符
  - 按位异或（^） 双目运算符
  - 取反（~） 单目运算符
  - 左移（<<） 单目运算符
  - 右移（>>） 单目运算符

位运算符（1）

- “按位与”运算符 &
  - 参加运算的两个数据，各位均独立进行“与”运算。
- 例如：对于表达式： a = 3 & 5，有：

$$3 = 00000011$$

$$(\&)\ 5 = 00000101$$

$$00000001$$

因此， 3 & 5 的值为：1

位运算符（2）

- “按位或”运算符 |
  - 参加运算的两个数据，各位均独立进行“或”运算。
- 例如：对于表达式： a = 3 | 5，有：

$$3 = 00000011$$

$$(|)\ 5 = 00000101$$

$$00000111$$

因此， 3 | 5 的值为：7

位运算符（3）

- “异或”运算符 ^
  - 异或运算符 ^ 也称 xor 运算符。参加运算的两个数，各位均独立进行 异或运算：

^	1	0
1	0	1
0	1	0

  - 例如：

```
    00111001   (十进制数57，八进制数071)
(^) 00101010   (十进制数42，八进制数052)
    00010011   (十进制数19，八进制数023)
```

- 即 071 ^ 052，结果为 023 (八进制数)。

位运算符（4）

- 取反运算符 ~
- 它是一个单目(元)运算符，用来对一个二进制数按位取反，
  - 即将 0 变 1 ，1 变 0 。
  - 例如：

000000000010101

(~) ↓

111111111101010

- 因此， ~025 的值为 0177752 。

位运算符（5）

- 左移运算符 <<
  - 用来将一个数的各二进位全部左移若干位。
  - 高位左移后溢出，舍弃不起作用。
  - 若 a 为 15，即二进制数 00001111，左移 2 位得 00111100，即十进制数 60 。

```
a = a << 2
```

  - 左移1位相当于该数乘以 2，左移 2 位相当于该数乘以 $2^2 = 4$ 。
    - 只适用于该数左移时被溢出舍弃的高位中不包含1的情况。

位运算符（6）

- 右移运算符 >>
- 用来将一个数的各二进位全部右移若干位。
  - 移到右端的低位被舍弃，对无符号数，高位补0。
  - 若 a 为 15，即二进制数 00001111，右移2位得 00000011，即十进制数 3；

```
a = a >> 2
```

  - 当没有非零数位被抛弃时，右移一位相当于除以 2，右移 n 位相当于除以 2^n 。

关于位运算的几个问题(1)

- 右移运算符号位的处理
  - 对无符号数，右移时左边高位移入 0 。
  - 对于有符号的值,
    - 若原来符号位为 0（正），则左边移入 0；
    - 若符号位原来为 1（负），则左边移入 0 还是 1，要取决于编译器和操作系统。
      - 若移入 0，称为“逻辑右移”或“简单右移”；
      - 若移入 1，称为“算术右移”。

关于位运算的几个问题(2)

- 不同长度的数据进行位运算
  - 如果两个数据长度不同进行位运算时，系统会将二者按右端对齐。
    - 如 a & b ，而 a 为 int 型，b 为 short 型
  - 如何补位
    - 如果 b 为无符号整数型，则左侧补满 0 。
    - 如果 b 为有符号整数型：
      - 如果 b 为正数，则左侧 16 位补满 0。
      - 如果 b 为负数，则左端 16 位补满 1。

关于位运算的几个问题(3)

- 位运算赋值运算符
  - 位运算符与赋值运算符可以组成复合赋值运算符如: &=　|=　>>=　<<=　^=
    - a &= b　　相当于　　a = a & b
    - a |= b　　相当于　　a = a | b
    - a >>= 2　　相当于　　a = a >> 2
    - a <<= 2　　相当于　　a = a << 2
    - a ^= b　　相当于　　a = a ^ b

关于位运算的几个问题(4)

优先级	运算符	说明	结合方向
2	a++ a-- type()	后缀自增与自减， 函数风格转型	→
3	++a --a ! ~ (type)	前缀自增与自减，**非**， C 风格转型	←
5	a*b a/b a%b	乘法、除法与取余	→
6	a+b a-b	加法与减法	→
7	<< >>	**左移与右移**	→
9	< <= > >=	（不）小于与大于	→
10	== !=	等于与不等于	→
11	**a&b**	**逐位与**	→
12	^	**逐位异或**	→
13	|	**逐位或**	→
14	&&	逻辑与	→
15	||	逻辑或	→
16	= += -= *= /= %= <<= >>= &= |= ^= a?b:c	简单与复合赋值，条件	←
17	,	逗号	→

注：条件运算符 ? 与 : 中间如同视有括号。

位运算的使用（1）

- “按位与”运算
  - 任意存储单元与0进行“按位与”运算可清零；
  - 取一个数中某些指定位
    - 如有一个整数a，想要其中的低字节。只需将a与 $(377)_8$ 按位与即可。

取低8位的值

a	00 10 11 00	10 10 11 00
b	00 00 00 00	11 11 11 11
c	00 00 00 00	10 10 11 00

- “按位或”运算
  - 对一个数据的某些位取定值为1；

位运算的使用（2）

- “异或”运算符的使用

(1) 使特定位翻转

- 使01111010低4位翻转（即1变为0，0变为1）可将其与00001111进行∧运算，即：

```
    01111010
(∧) 00001111
------------
    01110101
```

(2) 使特定位保持不变

- 与0相∧，保留原值如012∧00=012

```
    00001010
(∧) 00000000
------------
    00001010
```

“异或”运算符示例

- 交换两个值，而不必使用临时变量
  - 假如a = 3，b = 4。想将a和b的值互换，可以用以下赋值语句实现：

```
a = a ∧ b;
b = b ∧ a;
a = a ∧ b;
```

设：a = 3, b = 4

```
      a=011
(∧)   b=100
      ---------------------------
      a=111 (a∧b的结果，a已变成7)
(∧)   b=100
      ---------------------------
      b=011 (b∧a的结果，b已变成3)
(∧)   a=111
      ---------------------------
      a=100 (a∧b的结果，a变成4)
```

运算符示例

- 思考
  - 如何获得第一个比特位的数值

    利用 % 2　　利用 & 1
  - 如何以二进制显示

    利用 % 2 （&1）和 >> 1



思考：如何利用位运算解题

例题

- 判断一个数是否为2的N次方?

方法1:

```
while (num)
{
    if (num & 1 == 1)
        count++;
    num = num >> 1;
}
```

方法2:

```
if((num & (num-1)) == 0)
    return true;
else
    return false;
```



例题

- 输出所有可能子集
  - 输入 nums = [1, 2, 3]
  - 输出 [3] [1] [2] [1,2,3] [1,3] [2,3] [1,2] []
- 有32个房间，原本灯都是关的，小明把偶数房间的灯打开了，然后小红把20~32号房间的灯都关掉，小强再把15~22号房间的灯打开，小张把6 ~ 12的灯翻转。【不用数组可以吗】
  - 问：最后有多少个灯打开?



好好想想，有没有问题?

谢　谢　！